

Marjum 2026-07: replicating Christian's minimal Y-factor calibration

Question. Can we reproduce, from the campaign archive here, the calibration Christian sent on 2026-09-25 (`T12_yfactor_minimal.ipynb` + `d5_yfactor.py`)? It covers the night of 16/17 Jul 2026, input `4` (box-air), and gives

1. the three-state Y factor $T^* = T_{\mathrm{load}} + T_{\mathrm{NS}} \cdot (P_{\mathrm{ant}} - P_{\mathrm{load}}) / (P_{\mathrm{NS}} - P_{\mathrm{load}})$;
2. $T^* / (1 - |\Gamma_{\mathrm{ant}}|^2)$, the antenna S11 correction;
3. $T = [T^* - (1 - M_{\mathrm{load}}) T_{\mathrm{load}}] / M_{\mathrm{ant}}$ with $M_s = (1 - |\Gamma_s|^2) / (1 - |\Gamma_s \Gamma_{\mathrm{rec}}|^2)$: antenna, ambient-load and receiver S11 together.

How. The correlator side (loading, visit averaging, nearest-in-time pairing, T^* and the two corrections) runs Christian's own `d5_yfactor.py`, vendored unchanged next to this notebook. The S11 side is done by `eigsep_cal.S11` instead of his `calibrate_s11`, so this notebook also tests our package against his chain.

One setting matters: `SWITCH`. Two different lab characterizations of the RF-switch paths exist: Christian's `switch_sparams.npz` (file date 2026-08-31) and the one packaged in `eigsep_cal` (`switch_sparams2026.npz`). Their transmission terms agree to 0.3 %, but their port reflections differ by ~ 0.06 (up to ~ 0.15 at 250 MHz), and neither records its provenance. Which is current is being settled; until then the default is Christian's. § 6 shows what the choice does.

Answer. Yes. With `SWITCH = 'christian'` every number Christian's notebook prints is reproduced exactly (§ 2), and `eigsep_cal.S11` gives his $|\Gamma|$ at plane P to $\sim 1\text{e-}15$ (§ 6). With the packaged switch file, $|\Gamma|$ at P moves by up to 0.26 and the calibrated temperature by -6% to $+6\%$, largest near 65 and 240 MHz.

Status: diagnostic, as Christian's is. $T_{\mathrm{NS}} = 917\text{ K}$ is the nameplate ENR, not measured; no noise-wave, loss or beam correction; no RFI flagging, smoothing or time interpolation. The noise-source injection is known to drift 15 % over the night (`marjum-2026-07/INDEX.md`, calibration constraints); nearest-visit pairing tracks it only at the ~ 13 min cadence.

```
In [1]: # Provenance: repo commits and input hashes, so a rerun can be matched to this one.
import hashlib, subprocess, sys
from pathlib import Path

import eigsep_cal, eigsep_data, h5py, numpy
import cmt_vna

def git_rev(path):
    try:
        rev = subprocess.run(['git', '-C', str(path), 'rev-parse', '--short', 'HEAD'],
                              capture_output=True, text=True, check=True).stdout.strip()
        dirty = subprocess.run(['git', '-C', str(path), 'status', '--porcelain', '--', '.'],
                               capture_output=True, text=True).stdout.strip()
        branch = subprocess.run(['git', '-C', str(path), 'branch', '--show-current'],
                                 capture_output=True, text=True).stdout.strip()
        return f"{rev} [{branch}]" + (' DIRTY' if dirty else '')
    except Exception as e:
        return f'? ({e})'

def sha(path):
    return hashlib.sha256(Path(path).read_bytes()).hexdigest()[:16]

print('python', sys.version.split()[0], '| numpy', numpy.__version__,
      '| h5py', h5py.__version__, '| cmt_vna', cmt_vna.__version__)
for mod in (eigsep_cal, eigsep_data):
    print(f"{mod.__name__:12s}", git_rev(Path(mod.__file__).parent))
print(f"{'this_dir':12s}", git_rev(Path.cwd()))
print(f"{'d5_yfactor':12s} sha256 {sha('d5_yfactor.py')} (Christian's copy: 2471b6fab712...)")
```

```
python 3.11.11 | numpy 2.2.6 | h5py 3.14.0 | cmt_vna 1.4.1
eigsep_cal 3a98fe8 [main]
eigsep_data 97bceca [main]
this_dir 05e66eb [s11-cal-replication] DIRTY
d5_yfactor sha256 2471b6fab7128275 (Christian's copy: 2471b6fab712...)
```

0. Settings

Paths resolve through `eigsep_data`'s campaign root (`EIGSEP_CAMPAIGN_ROOT`, else the sibling `marjum-2026-07/` checkout), so the notebook runs unchanged on limbo and the laptop. The window and key are Christian's.

Christian's two lab files are tracked in `marjum-2026-07/data/s11/cal_materials/` (copied unchanged from his bundle). The internal-OSL characterization always comes from `eigsep_cal`'s packaged file; § 6 shows it equals the characterization of Christian's raw lab capture to $\sim 1e-15$.

```

In [2]: from datetime import datetime
        from zoneinfo import ZoneInfo

import matplotlib.colors as mcolors
import matplotlib.dates as mdates
import matplotlib.pyplot as plt
import numpy as np
import pandas as pd
import eigsep_cal.s11
from eigsep_cal import S11

import d5_yfactor as yf

CAMPAIGN = eigsep_data.get_campaign_root(
    default=Path('.././.././../marjum-2026-07').resolve())
CORR_DIR = CAMPAIGN / 'data'
S11_DIR = CAMPAIGN / 'data' / 's11'
CAL_MATERIALS = S11_DIR / 'cal_materials'
PACKAGED = eigsep_cal.s11.PACKAGED_FILES[2026]

#: Which switch-path S-parameters take Gamma from the VNA plane to plane P.
SWITCH_FILES = {
    'christian': CAL_MATERIALS / 'switch_sparams.npz',      # his bundle, 2026-08-31
    'eigsep_cal': PACKAGED['switchpaths'],                  # switch_sparams2026.npz
}
SWITCH = 'christian'
INTERNAL_OSL = PACKAGED['osldata']                          # characterized O/S/L
INTERNAL_OSL_RAW = CAL_MATERIALS / 'vna_internal_osl_20260629T090825Z.npz' # $ 6 only

TZ = ZoneInfo('America/Denver')
START = datetime(2026, 7, 16, 21, 30, tzinfo=TZ) # the clean night of 16/17 Jul
STOP = datetime(2026, 7, 17, 7, 55, tzinfo=TZ)
KEY = '4' # box-air, the switched input

PLOT_FREQ_MHZ = (50, 250) # display only
C_STAGE = ['#2a78d6', '#eb6834', '#1baf7a'] # T*, +ant S11, +ant+rec S11
C_SWITCH = {'christian': '#2a78d6', 'eigsep_cal': '#eb6834'}

print('campaign root:', CAMPAIGN)
print(f'{START:%Y-%m-%d %H:%M} to {STOP:%Y-%m-%d %H:%M} MDT, key {KEY!r}')
print(f'T_NS = {yf.T_NS:.1f} K (assumed)')
print(f'SWITCH = {SWITCH!r}: {SWITCH_FILES[SWITCH].name} sha256 {sha(SWITCH_FILES[SWITCH])}')
print(f'internal OSL: {INTERNAL_OSL.name} sha256 {sha(INTERNAL_OSL)}')

```

```

campaign root: /mnt/data02/eigsep/marjum-2026-07
2026-07-16 21:30 to 2026-07-17 07:55 MDT, key '4'
T_NS = 917.1 K (assumed)
SWITCH = 'christian': switch_sparams.npz sha256 8b4418f07187c965
internal OSL: calibrated_internal_osl2026.npz sha256 832ba837e98017f5

```

```
In [3]: # Plot helpers.
def as_num(t):
    return mdates.date2num((np.asarray(t) * 1e6).astype('datetime64[us]'))

def time_axis(axis, label=True):
    axis.set_major_locator(mdates.HourLocator(interval=2, tz=TZ))
    axis.set_major_formatter(mdates.DateFormatter('%H:%M', tz=TZ))
    if label:
        axis.set_label_text('local time (MDT)')

def band(freqs, lo_hi):
    return (freqs >= lo_hi[0]) & (freqs < lo_hi[1])

def waterfall(ax, visits, values, freqs, label):
    # Each visit spans its own [t_start, t_stop]; gaps between visits stay blank.
    edges = as_num(np.column_stack([visits['t_start'], visits['t_stop']]).ravel())
    z = np.full((2 * len(values) - 1, values.shape[1]), np.nan)
    z[::2] = values
    df = freqs[1] - freqs[0]
    f_edges = np.r_[freqs - df / 2, freqs[-1] + df / 2]
    v = values[:, band(freqs, PLOT_FREQ_MHZ)]
    v = v[np.isfinite(v) & (v > 0)]
    mesh = ax.pcolormesh(f_edges, edges, z, norm=mcolors.LogNorm(*np.percentile(v, [1, 99])),
                        shading='flat', rasterized=True, cmap='viridis')
    ax.set(xlim=PLOT_FREQ_MHZ, ylim=(edges[-1], edges[0]), xlabel='frequency (MHz)')
    time_axis(ax.yaxis)
    ax.figure.colorbar(mesh, ax=ax, label=label)

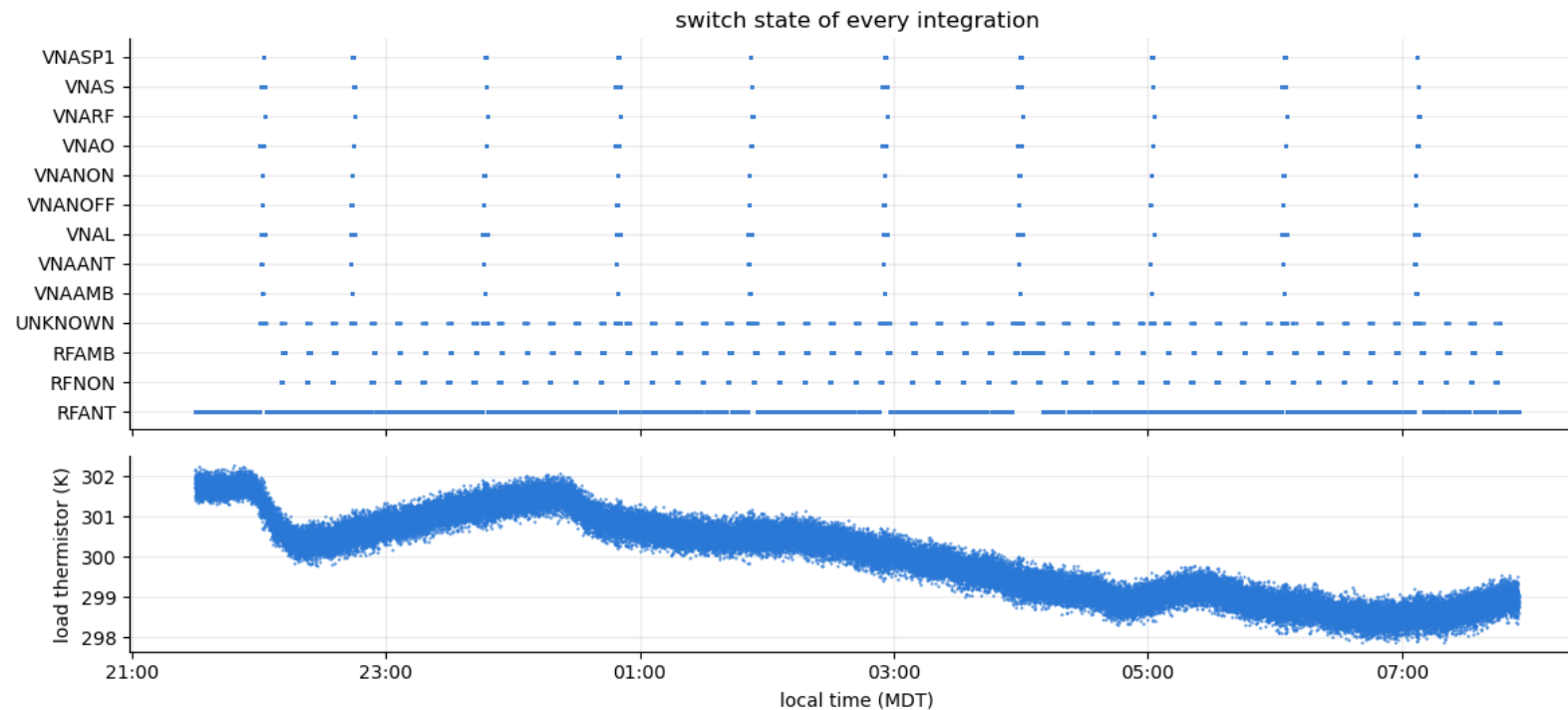
plt.rcParams.update({'axes.grid': True, 'grid.alpha': 0.25, 'lines.linewidth': 1.2,
                    'axes.spines.top': False, 'axes.spines.right': False})
```

1. Load the correlator data

`yf.load_corr` reads every integration of `KEY` in the window: header time (or filename-derived where the clock was stale), `rfswitch` state, and load thermistor `tempctrl_load.T_now`. Nothing is dropped.

```
In [4]: d = yf.load_corr(CORR_DIR, START.timestamp(), STOP.timestamp(), KEY)
freqs = d['freqs']
zero = np.all(d['spec'] == 0, axis=1)
wrapped = d['spec'] < 0

fig, (ax0, ax1) = plt.subplots(2, 1, figsize=(12, 5.5), sharex=True, height_ratios=(2, 1))
states = [yf.SKY, yf.NOISE, yf.LOAD] + sorted(set(d['state']) - {yf.SKY, yf.NOISE, yf.LOAD})
ax0.scatter(as_num(d['time']), [states.index(s) for s in d['state']], s=1,
            color=C_STAGE[0], rasterized=True)
ax0.set_yticks(range(len(states)), states)
ax0.set_title('switch state of every integration')
ax1.plot(as_num(d['time']), d['t_load'], '.', ms=1, color=C_STAGE[0])
ax1.set_ylabel('load thermistor (K)')
time_axis(ax1.xaxis)
fig.tight_layout()
```



2. Visits, pairing, and the check against Christian's printed numbers

A visit is a contiguous run of one switch state, reduced to its **median** spectrum (int32 wraps in a few RFI channels would bias a mean). All-zero rows are the only exclusion. Each sky visit then takes the nearest load and noise-source visits in time, with no age limit.

Christian's notebook (run 2026-09-25 on his laptop, filtered D5 set) printed the values in the `christian` column; ours are computed here from the campaign archive.

```

In [5]: visits = yf.average_blocks(d, use=~zero)
sky, noise, load = (visits[s] for s in (yf.SKY, yf.NOISE, yf.LOAD))
i_load, age_load = yf.nearest(sky['t'], load['t'])
i_noise, age_noise = yf.nearest(sky['t'], noise['t'])
P_sky, P_load, P_noise = sky['spec'], load['spec'][i_load], noise['spec'][i_noise]
T_load = load['t_load'][i_load]

ours = {
    'integrations': len(d['time']),
    'files': len(set(d['file'])),
    'rows timed from filename': int(d['time_from_filename'].sum()),
    'all-zero rows': int(zero.sum()),
    'rows with a wrapped value': int(wrapped.any(axis=1).sum()),
    'channels with a wrapped value': int(wrapped.any(axis=0).sum()),
    'rows without thermistor': int(np.isnan(d['t_load']).sum()),
    'RFANT / RFNON / RFAMB rows': '/'.join(str((d['state'] == s).sum()) for s in (yf.SKY, yf.NOISE, yf.LOAD)),
    'RFANT / RFNON / RFAMB visits': '/'.join(str(len(visits[s]['t'])) for s in (yf.SKY, yf.NOISE, yf.LOAD)),
    'largest load age (min)': round(age_load.max() / 60, 1),
    'largest noise age (min)': round(age_noise.max() / 60, 1),
    'T_load min (K)': round(np.nanmin(T_load), 2),
    'T_load max (K)': round(np.nanmax(T_load), 2),
}
christian = {
    'integrations': 69849, 'files': 292, 'rows timed from filename': 0,
    'all-zero rows': 0, 'rows with a wrapped value': 289,
    'channels with a wrapped value': 16, 'rows without thermistor': 190,
    'RFANT / RFNON / RFAMB rows': '55459/5282/6096',
    'RFANT / RFNON / RFAMB visits': '54/48/49',
    'largest load age (min)': 26.5, 'largest noise age (min)': 25.6,
    'T_load min (K)': 298.46, 'T_load max (K)': 301.53,
}
check = pd.DataFrame({'ours': ours, 'christian': christian})
check['match'] = [str(a) == str(b) for a, b in zip(check.ours, check.christian)]
assert check.match.all(), check[~check.match]
check

```

Out[5]:

	ours	christian	match
integrations	69849	69849	True
files	292	292	True
rows timed from filename	0	0	True
all-zero rows	0	0	True
rows with a wrapped value	289	289	True
channels with a wrapped value	16	16	True
rows without thermistor	190	190	True
RFANT / RFNON / RFAMB rows	55459/5282/6096	55459/5282/6096	True
RFANT / RFNON / RFAMB visits	54/48/49	54/48/49	True
largest load age (min)	26.5	26.5	True
largest noise age (min)	25.6	25.6	True
T_load min (K)	298.46	298.46	True
T_load max (K)	301.53	301.53	True

3. Three-state Y factor

The uncalibrated antenna temperature at the receiver input. The bright dashes near 137 MHz are ORBCOMM passes during single sky visits.

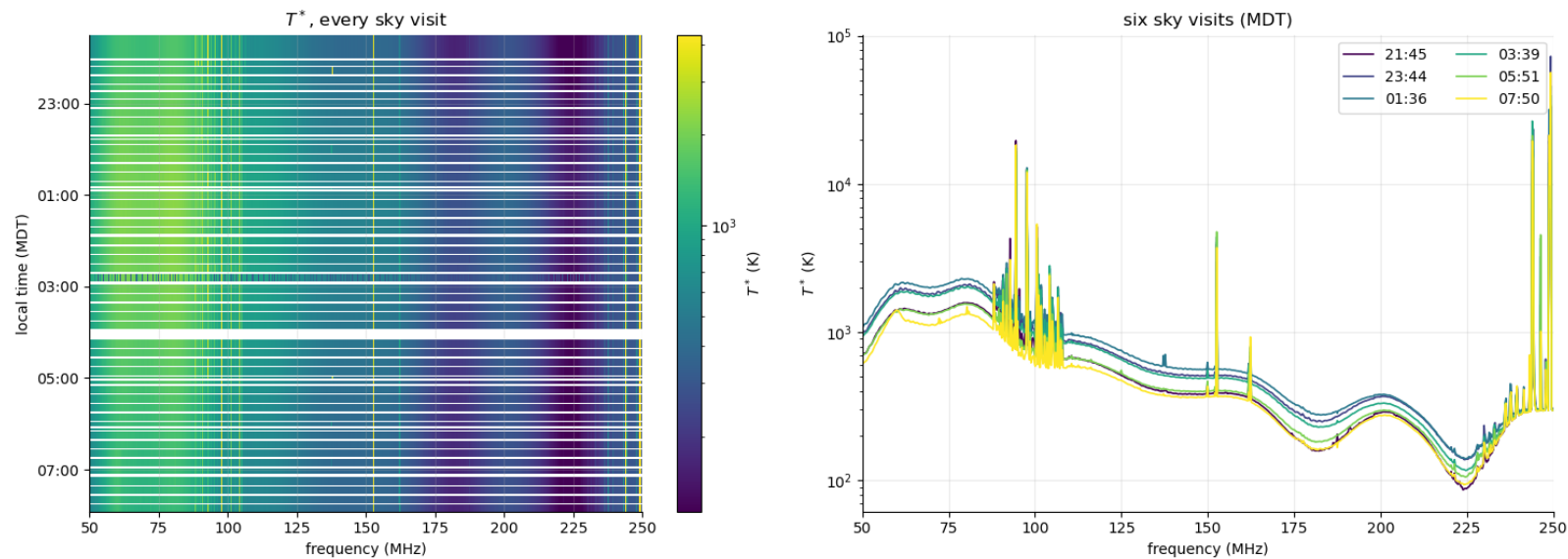
```
In [6]: T_star = yf.tant_star(P_sky, P_noise, P_load, T_load[:, None])

fig, (ax0, ax1) = plt.subplots(1, 2, figsize=(15, 5.5))
waterfall(ax0, sky, T_star, freqs, r'$T^{*}$ (K)')
ax0.set_title(r'$T^{*}$, every sky visit')
cmap = plt.get_cmap('viridis')
pick = np.linspace(0, len(sky['t']) - 1, 6).astype(int)
for j, i in enumerate(pick):
    t = datetime.fromtimestamp(sky['t'][i], TZ)
    ax1.semilogy(freqs, np.where(T_star[i] > 0, T_star[i], np.nan),
                 color=cmap(j / (len(pick) - 1)), label=f'{t:%H:%M}')
ax1.set(xlim=PLOT_FREQ_MHZ, xlabel='frequency (MHz)', ylabel=r'$T^{*}$ (K)',
        title='six sky visits (MDT)')
ax1.legend(ncol=2)
fig.tight_layout()

def loglog_slope(T, lo, hi):
    k = band(freqs, (lo, hi))
    return np.polyfit(np.log(freqs[k]), np.log(np.median(T[:, k], axis=0)), 1)[0]

print('log-log slope of the median T*: '
      f'{loglog_slope(T_star, 50, 88):+.2f} (50-88 MHz), '
      f'{loglog_slope(T_star, 108, 136):+.2f} (108-136 MHz)')
```

log-log slope of the median T*: +0.83 (50-88 MHz), -2.46 (108-136 MHz)



Christian's README quotes T^* slopes of +0.82 (50–88 MHz) and -2.45 (108–136 MHz) for this night, computed outside his notebook with an unstated fit. A straight log-log fit to the median spectrum gives the values above (+0.83, -2.46); a fit to the mean spectrum gives +0.82 and -2.38 . That is close, not an exact check; the exact checks are § 2 and § 6.

4. S11 at plane P with `eigsep_cal.S11`

`S11(S11_DIR).calibrate(switchpaths, osldata)` runs, per sweep:

1. the VNA error network, solved from the sweep's internal O/S/L traces against their characterization, and removed (`vna` plane);
2. the VNA-leg switch path de-embedded (`dut` plane);
3. the RF-leg switch path embedded (`lna` plane = P). The receiver has no RF leg and stops at `dut`.

A sweep whose own internal OSL is zero-filled (cmt_vna #54) borrows the nearest valid one of the same mode, as Christian's chain does; zero-filled DUT traces are skipped. `default` is the deepest plane each DUT reaches. The antenna γ includes the balun and feed coax.

```

In [7]: def s11_at_p(switch):
s = S11(S11_DIR).calibrate(SWITCH_FILES[switch], INTERNAL_OSL)
out = {}
for dut in ('ant', 'amb', 'rec'):
    t, g = s.get_all_s11s(dut, 'default')
    out[dut] = {'t': np.asarray(t, float), 'gamma': np.asarray(g)}
return s.freqs, out, s

freqs_s11, s11, s11_obj = s11_at_p(SWITCH)
print('skipped files:', s11_obj.skipped or 'none')
print({dut: len(v['t']) for dut, v in s11.items()}, 'calibrated sweeps (all 40 files)')

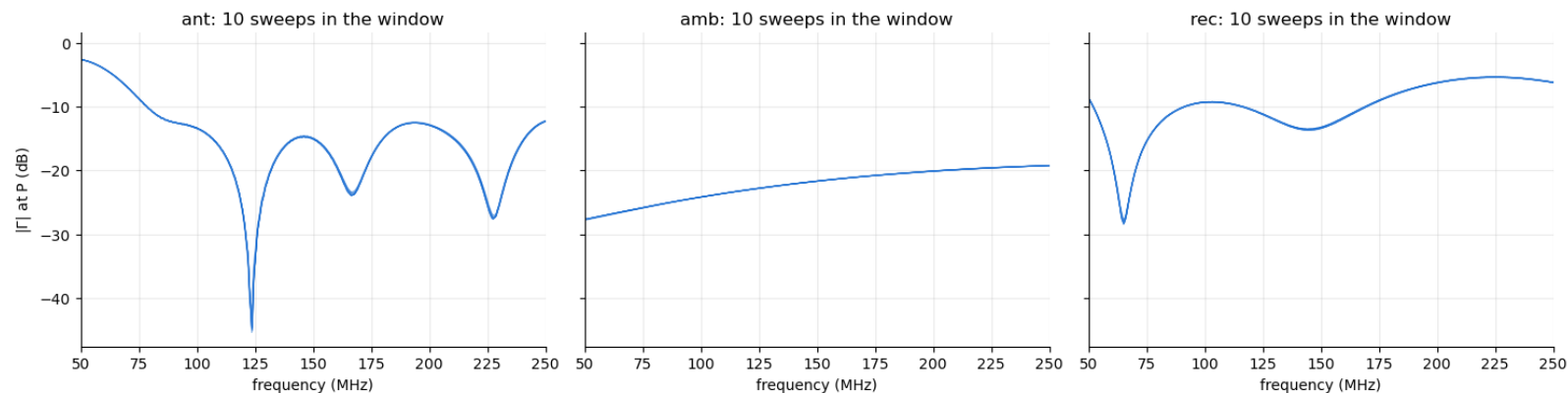
fig, axes = plt.subplots(1, 3, figsize=(15, 4.2), sharey=True)
win = lambda t: (t >= START.timestamp()) & (t < STOP.timestamp())
for ax, dut in zip(axes, ('ant', 'amb', 'rec')):
    keep = win(s11[dut]['t'])
    for g in s11[dut]['gamma'][keep]:
        ax.plot(freqs_s11 / 1e6, 20 * np.log10(np.abs(g)), lw=0.8, color=C_STAGE[0], alpha=0.6)
    ax.set(xlim=PLOT_FREQ_MHZ, xlabel='frequency (MHz)',
           title=f'{dut}: {keep.sum()} sweeps in the window')
axes[0].set_ylabel(r'$|\Gamma|_{at P}$ (dB)')
fig.suptitle(f'SWITCH = {SWITCH!r}')
fig.tight_layout()

```

skipped files: none

{'ant': 18, 'amb': 18, 'rec': 20} calibrated sweeps (all 40 files)

SWITCH = 'christian'



5. The two S11 corrections

Each sky visit takes the antenna, ambient-load and receiver sweeps nearest in time (largest ages printed). The 1000-point S11 grid goes onto the correlator channels by linear interpolation of real and imaginary parts. The receiver-S11 correction assumes equal gain on every switch path, no receiver noise waves, and T_{NS} as the excess delivered to the receiver.

```

In [8]: def corrections(s11):
    gam, age = {}, {}
    for dut in ('ant', 'amb', 'rec'):
        i, age[dut] = yf.nearest(sky['t'], s11[dut]['t'])
        gam[dut] = yf.s11_to_channels(s11[dut]['gamma'][i], freqs_s11 / 1e6, freqs)
    T_ant = yf.correct_antenna_s11(T_star, gam['ant'])
    T_rec = yf.correct_receiver_s11(T_star, T_load[:, None], gam['ant'], gam['amb'], gam['rec'])
    return gam, age, T_ant, T_rec

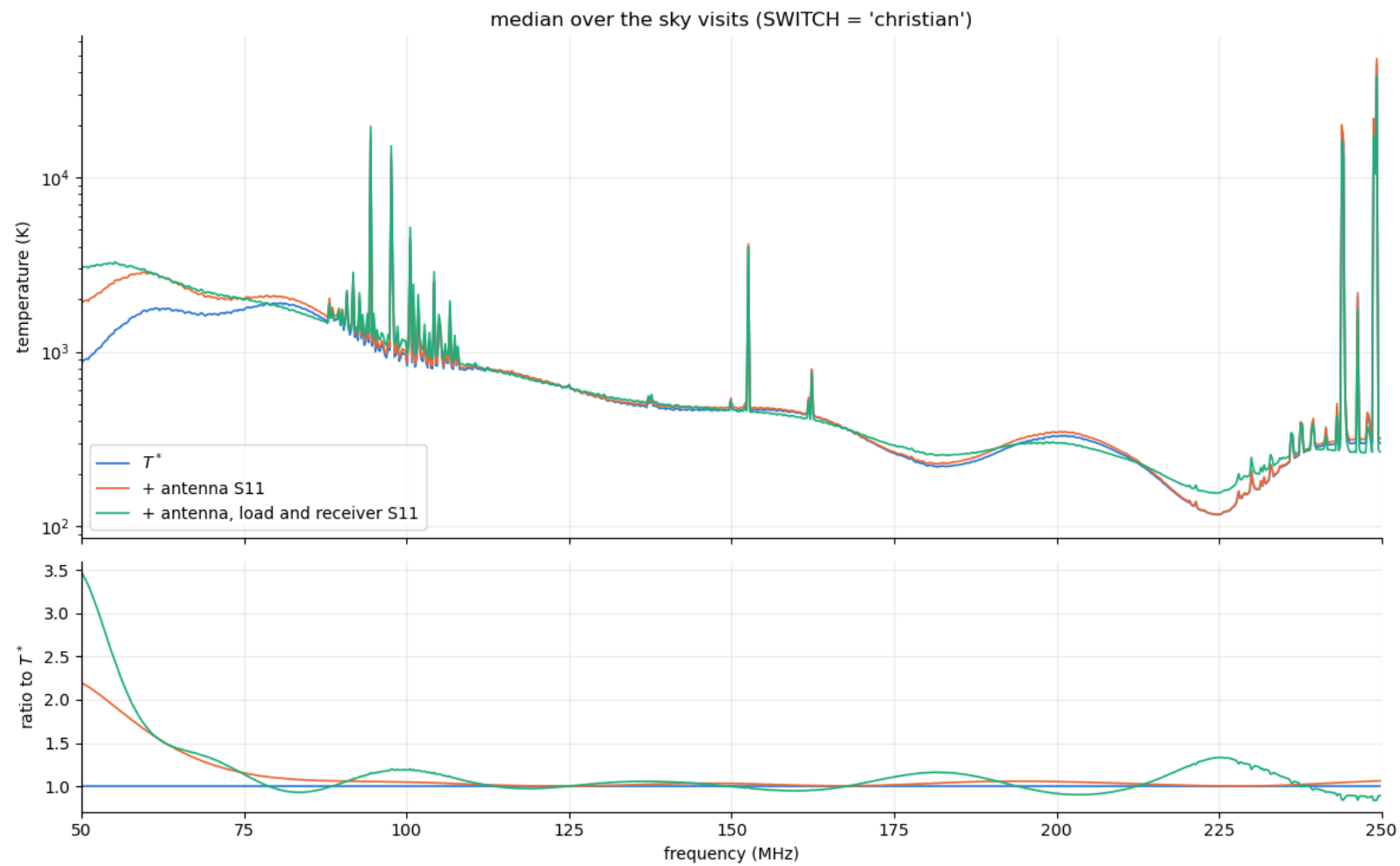
gam, age_s11, T_ant_s11, T_rec_s11 = corrections(s11)
print('largest S11 age (min):', {k: round(v.max() / 60, 1) for k, v in age_s11.items()},
      '| Christian: ant 31.6, amb 31.6, rec 30.8')

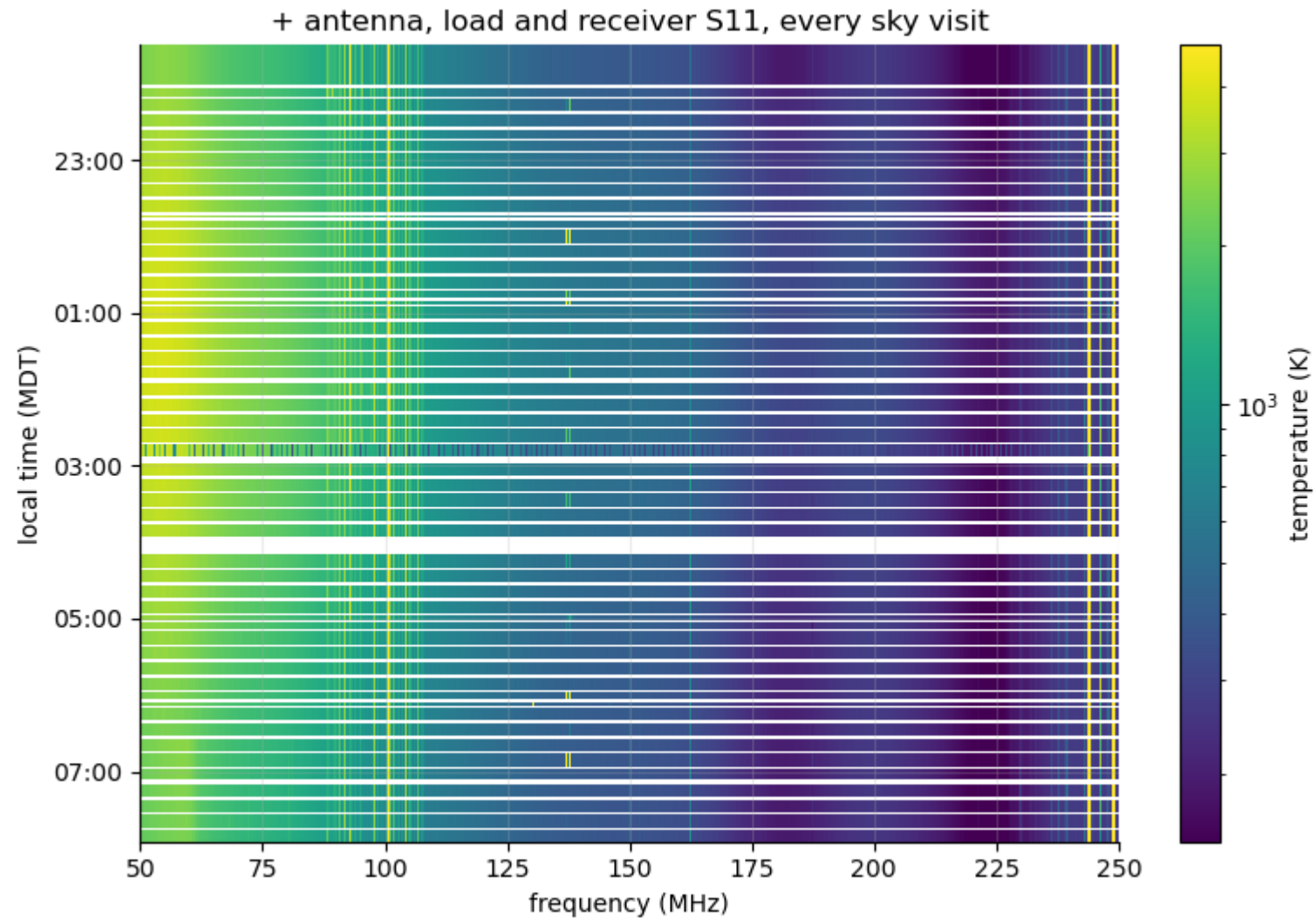
f = band(freqs, PLOT_FREQ_MHZ)
fig, (ax0, ax1) = plt.subplots(2, 1, figsize=(12, 7.5), sharex=True, height_ratios=(2, 1))
for arr, label, c in zip((T_star, T_ant_s11, T_rec_s11),
                          (r'$T^{*}$', '+ antenna S11', '+ antenna, load and receiver S11'), C_STAGE):
    med = np.median(arr[:, f], axis=0)
    ax0.semilogy(freqs[f], np.where(med > 0, med, np.nan), color=c, label=label)
    ax1.plot(freqs[f], np.median(arr[:, f] / T_star[:, f], axis=0), color=c)
ax0.set(ylabel='temperature (K)', title=f'median over the sky visits (SWITCH = {SWITCH!r})')
ax0.legend()
ax1.set(xlim=PLOT_FREQ_MHZ, xlabel='frequency (MHz)', ylabel=r'ratio to $T^{*}$')
fig.tight_layout()

fig, ax = plt.subplots(figsize=(8, 5.5))
waterfall(ax, sky, T_rec_s11, freqs, 'temperature (K)')
ax.set_title('+ antenna, load and receiver S11, every sky visit')
fig.tight_layout()

```

largest S11 age (min): {'ant': np.float64(31.6), 'amb': np.float64(31.6), 'rec': np.float64(30.8)} | Christian: ant 31.6, amb 31.6, rec 30.8





6. Cross-checks

(a) `eigsep_cal.S11` against Christian's `calibrate_s11`. His function characterizes the internal OSL from the raw lab capture itself (`vna_internal_osl_20260629T090825Z.npz` , against the S911T kit model) and uses his switch file. Run on the same 40 sweeps, both chains should agree to rounding if `SWITCH = 'christian'`.

```
In [9]: _, osl_from_raw = yf.characterize_internal_osl(INTERNAL_OSL_RAW)
o = np.load(INTERNAL_OSL)
print('internal-OSL characterization, eigsep_cal packaged vs from Christian raw capture: '
      f'max|d| = {np.abs(np.array([o["O"], o["S"], o["L"]]) - osl_from_raw).max():.1e}')

_, s11_ref, log = yf.calibrate_s11(S11_DIR, INTERNAL_OSL_RAW, SWITCH_FILES['christian'])
_, s11_chr, _ = s11_at_p('christian')
rows = []
for dut in ('ant', 'amb', 'rec'):
    same_t = np.array_equal(s11_ref[dut]['t'], s11_chr[dut]['t'])
    rows.append({'dut': dut, 'sweeps (Christian)': len(s11_ref[dut]['t']),
                'sweeps (eigsep_cal)': len(s11_chr[dut]['t']), 'same times': same_t,
                'max |dGamma|': np.abs(s11_ref[dut]['gamma'] - s11_chr[dut]['gamma']).max()
                if same_t else np.nan})
xa = pd.DataFrame(rows).set_index('dut')
assert (xa['max |dGamma|'] < 1e-12).all()
xa.style.format({'max |dGamma|': '{:.1e}'})
```

internal-OSL characterization, eigsep_cal packaged vs from Christian raw capture: max|d| = 9.0e-16

```
Out[9]:
```

	sweeps (Christian)	sweeps (eigsep_cal)	same times	max dGamma
dut				
ant	18	18	True	1.4e-15
amb	18	18	True	8.9e-16
rec	20	20	True	1.4e-15

(b) What the switch file does. Both characterizations run through the same chain and the same corrections; only SWITCH_FILES differs.

In [10]:

```
res = {}
for sw in SWITCH_FILES:
    _, s, _ = s11_at_p(sw)
    res[sw] = (s, *corrections(s))

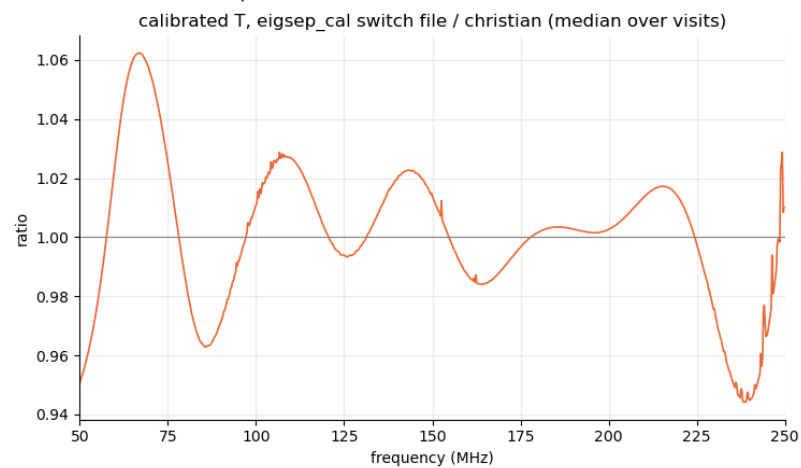
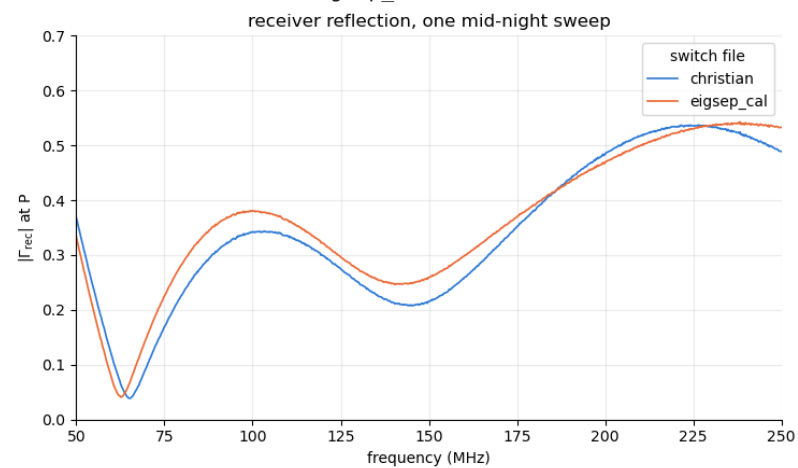
s_c, s_e = res['christian'][0], res['eigsep_cal'][0]
fs = freqs_s11 / 1e6
bs = (fs >= 50) & (fs < 200)
tab = pd.DataFrame({dut: {
    'max |dGamma| at P': np.abs(s_c[dut]['gamma'] - s_e[dut]['gamma']).max(),
    'mean |Gamma| 50-200 MHz, christian': np.abs(s_c[dut]['gamma'][:, bs]).mean(),
    'mean |Gamma| 50-200 MHz, eigsep_cal': np.abs(s_e[dut]['gamma'][:, bs]).mean(),
} for dut in ('ant', 'amb', 'rec')}).T
display(tab.round(3))

with np.errstate(invalid='ignore', divide='ignore'):
    ratio = res['eigsep_cal'][4] / res['christian'][4] # T_rec_s11
for lo, hi in [(50, 88), (88, 108), (108, 136), (136, 200), (200, 250)]:
    k = band(freqs, (lo, hi))
    r = np.median(ratio[:, k], axis=0)
    print(f'T (ant+load+rec S11), eigsep_cal / christian, {lo:3d}-{hi:3d} MHz: '
          f'median over visits spans {r.min():.3f} .. {r.max():.3f}')

fig, (ax0, ax1) = plt.subplots(1, 2, figsize=(15, 4.5))
i_mid = np.argmin(np.abs(s_c['rec']['t'] - 0.5 * (START.timestamp() + STOP.timestamp())))
for sw, c in C_SWITCH.items():
    ax0.plot(fs, np.abs(res[sw][0]['rec']['gamma'][i_mid]), color=c, label=sw)
ax0.set(xlim=PLOT_FREQ_MHZ, ylim=(0, 0.7), xlabel='frequency (MHz)', ylabel=r'$|\Gamma_{\mathrm{rec}}|$ at P',
        title='receiver reflection, one mid-night sweep')
ax0.legend(title='switch file')
ax1.plot(freqs[f], np.median(ratio[:, f], axis=0), color=C_SWITCH['eigsep_cal'])
ax1.axhline(1, color='0.5', lw=0.8)
ax1.set(xlim=PLOT_FREQ_MHZ, xlabel='frequency (MHz)', ylabel='ratio',
        title='calibrated T, eigsep_cal switch file / christian (median over visits)')
fig.tight_layout()
```

	max dGamma at P	mean Gamma 50-200 MHz, christian	mean Gamma 50-200 MHz, eigsep_cal
ant	0.012	0.233	0.234
amb	0.013	0.072	0.075
rec	0.260	0.305	0.330

T (ant+load+rec S11), eigsep_cal / christian, 50- 88 MHz: median over visits spans 0.951 .. 1.062
T (ant+load+rec S11), eigsep_cal / christian, 88-108 MHz: median over visits spans 0.966 .. 1.029
T (ant+load+rec S11), eigsep_cal / christian, 108-136 MHz: median over visits spans 0.993 .. 1.027
T (ant+load+rec S11), eigsep_cal / christian, 136-200 MHz: median over visits spans 0.984 .. 1.023
T (ant+load+rec S11), eigsep_cal / christian, 200-250 MHz: median over visits spans 0.944 .. 1.029



7. What you have

On native correlator channels `freqs` (MHz), one row per sky visit (`sky['t']` , `sky['t_start']` , `sky['t_stop']` in unix s):

Array	Contents
<code>P_sky</code> , <code>P_load</code> , <code>P_noise</code>	visit spectra (counts): sky and its nearest load / noise-source visits
<code>T_load</code>	thermistor temperature of the nearest load visit (K)
<code>T_star</code>	three-state Y factor (K)
<code>gam['ant']</code> , <code>gam['amb']</code> , <code>gam['rec']</code>	nearest-in-time γ at P, from <code>SWITCH</code>
<code>T_ant_s11</code> , <code>T_rec_s11</code>	<code>T_star</code> with the antenna, and antenna + load + receiver, S11 corrections
<code>res[switch]</code>	(<code>s11</code> , <code>gam</code> , <code>age</code> , <code>T_ant_s11</code> , <code>T_rec_s11</code>) for each switch file
<code>age_load</code> , <code>age_noise</code> , <code>age_s11</code>	time from each sky visit to what it was paired with (s)

Not settled here: which switch characterization is current; T_{NS} (nameplate, and the injection drifts through the night); any noise-wave or loss term.